Mantenimiento predictivo de motores turbofan: estimacion de la vida util remanente (RUL) con NASA C-MAPSS FD001

Reporte tecnico de analitica aeronautica
Asignatura: Desarrollo de Algoritmos para la Simulacion de Sistemas
Carrera: Ingenieria en Aeronautica
Autor(es): Completar nombre(s) del equipo
Fecha: noviembre de 2026

Resumen

Se desarrollo un pipeline reproducible que transforma la telemetria sintetica de 100 motores turbofan (NASA C-MAPSS, subconjunto FD001) en una estimacion de la vida util remanente (RUL). Tras la limpieza y la eliminacion de 6 sensores constantes, se compararon tres modelos con validacion agrupada por motor. El mejor, un Random Forest ajustado, obtuvo en la evaluacion oficial un RMSE de 17.93 ciclos, un MAE de 12.85 ciclos y un R2 de 0.814. Los resultados se traducen en una propuesta de umbrales de alerta para programar inspecciones, y se entrega un tablero interactivo en Streamlit.

1. Problema y justificacion operativa

El mantenimiento programado por intervalos fijos puede retirar componentes con vida util aun disponible o, en el extremo opuesto, no detectar a tiempo una degradacion acelerada. El mantenimiento predictivo (PdM) busca estimar cuantos ciclos de vuelo le quedan a un motor a partir de sus sensores, de modo que la inspeccion se programe segun su condicion real.

El problema planteado es de regresion: dado el historial de sensores de un motor hasta el ciclo actual, estimar la RUL, es decir, los ciclos restantes antes de la falla. C-MAPSS es un conjunto sintetico de la NASA (Prognostics Center of Excellence) que simula la degradacion run-to-failure de motores turbofan, y es una referencia habitual para validar metodos de pronostico.

2. Datos

El subconjunto FD001 contiene una sola condicion operativa y un solo modo de falla. El conjunto de entrenamiento tiene 20631 registros de 100 motores; el de prueba, 13096 registros de otros 100 motores cuya RUL real se entrega en un archivo aparte (RUL_FD001.txt). Cada registro incluye el numero de motor, el ciclo, tres ajustes operacionales y 21 sensores.

La vida de los motores de entrenamiento varia entre 128 y 362 ciclos (media 206), lo que se aprecia en la Figura 1. Esta dispersion implica que no basta con un umbral fijo de ciclos para decidir el mantenimiento.

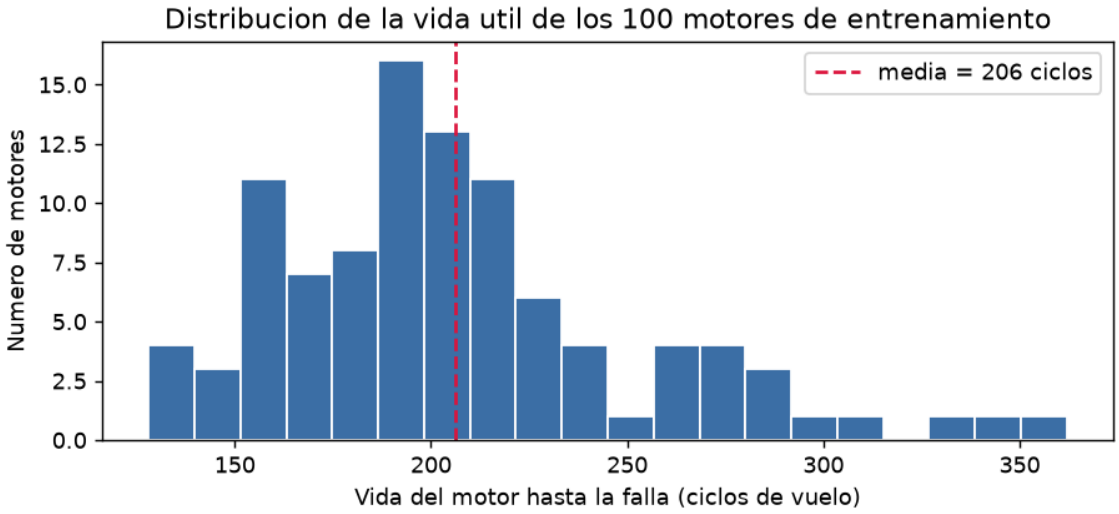


Figura 1. Distribucion de la vida de los motores de entrenamiento.

3. Metodologia de limpieza y validacion de consistencia

Nulos y duplicados. La auditoria no encontro valores nulos ni filas duplicadas en entrenamiento ni en prueba. Aun asi, los modelos incluyen un imputador por mediana dentro del Pipeline, de modo que se ajusta solo con datos de entrenamiento y se evita fuga de informacion (data leakage).

Sensores constantes. Se eliminaron sensor_1, sensor_5, sensor_10, sensor_16, sensor_18, sensor_19, porque su valor no cambia a lo largo de la vida del motor y no aportan informacion discriminante en FD001. Quedaron 15 sensores como variables de entrada.

Valores atipicos. Se marcaron 5406 observaciones fuera del rango intercuartilico (IQR), con mayor concentracion en sensor_9 (1686) y sensor_14 (1543). No se eliminaron: en un ensayo hasta la falla, los valores extremos suelen corresponder a los ultimos ciclos, es decir, a la degradacion real que se

quiere detectar. Borrarlos eliminaria justamente la senal de interes.

Unidades. C-MAPSS entrega ajustes y sensores en escalas propias del simulador, sin unidades SI verificadas, por lo que no se aplicaron conversiones para no inventarlas. La estandarizacion se aplica dentro de los pipelines solo cuando el modelo lo requiere.

Variable objetivo. La RUL de entrenamiento se calcula como el ultimo ciclo del motor menos el ciclo actual, y se limita a 125 ciclos (transformacion habitual en C-MAPSS): mientras el motor esta sano, la degradacion es poco observable y distinguir 200 de 150 ciclos restantes no tiene valor operativo.

4. Analisis exploratorio y estadistica inferencial

La Figura 2 muestra la evolucion de los cuatro sensores con mayor asociacion a la RUL, en cinco motores. Se observan tendencias monotonas hacia la falla que se hacen mas pronunciadas en los ultimos ciclos, lo que es consistente con un proceso de degradacion progresiva.

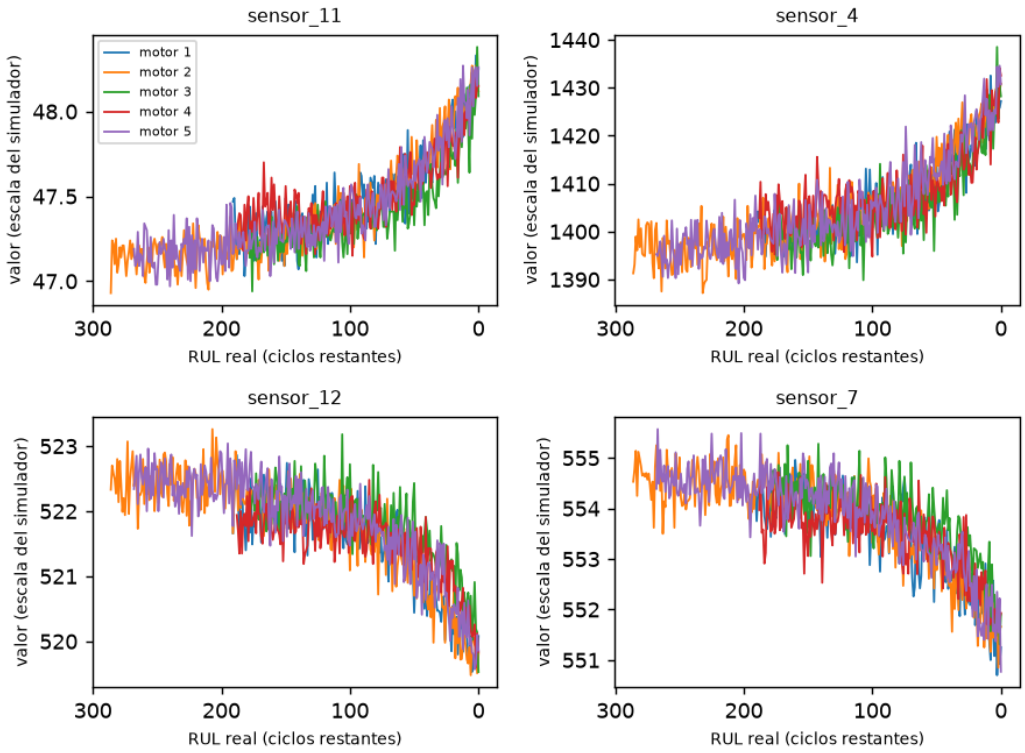


Figura 2. Trayectorias de sensor_11, sensor_4, sensor_12, sensor_7 frente a la RUL real (cinco motores).

Como las relaciones pueden ser no lineales, se uso la correlacion de Spearman (monotona) entre cada sensor y la RUL (Figura 3). Ademas se formularon las hipotesis: H0, la distribucion del sensor es la misma en las tres etapas de vida (sana: RUL mayor a 100 ciclos; intermedia: RUL entre 50 y 100; critica: RUL de 50 o menos); H1, al menos una etapa difiere. Se contrasto con ANOVA de un factor y con la prueba no parametrica de Kruskal-Wallis, que no exige normalidad.

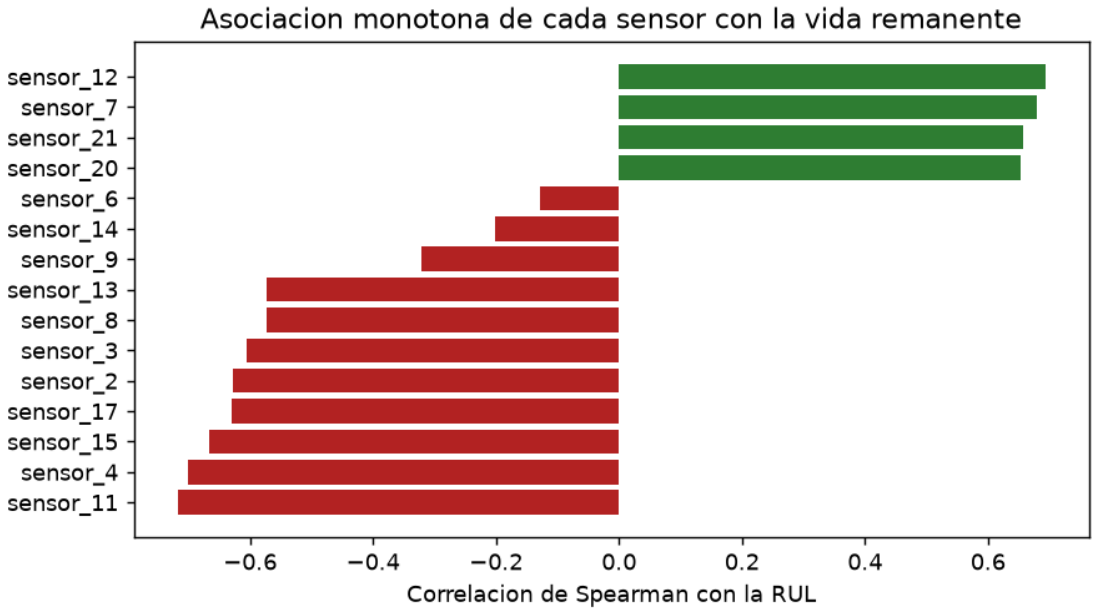


Figura 3. Correlacion de Spearman entre sensores y RUL.

Sensor	rho Spearman	p (rho)	F ANOVA	p (ANOVA)	H Kruskal	p (Kruskal)
sensor_11	-0.718	0.0e+00	12168	0.0e+00	10454	0.0e+00
sensor_4	-0.702	0.0e+00	11136	0.0e+00	10058	0.0e+00
sensor_12	0.693	0.0e+00	10672	0.0e+00	9851	0.0e+00
sensor_7	0.679	0.0e+00	9760	0.0e+00	9466	0.0e+00
sensor_15	-0.666	0.0e+00	9078	0.0e+00	9157	0.0e+00
sensor_21	0.657	0.0e+00	8510	0.0e+00	8863	0.0e+00
sensor_20	0.653	0.0e+00	8369	0.0e+00	8804	0.0e+00
sensor_17	-0.629	0.0e+00	7436	0.0e+00	8247	0.0e+00

Para los 8 sensores con mayor |rho|, todos los p-valores son muy inferiores a 0.05, por lo que se rechaza H0: la distribucion de los sensores cambia significativamente con la etapa de vida. Esto justifica usarlos como predictores. Con 20631 observaciones, la significancia estadistica es facil de alcanzar; por eso se interpreta junto con el tamano del efecto (rho), no solo con el p-valor. Ademas, las observaciones de un mismo motor no son independientes entre si, de modo que los p-valores son orientativos.

5. Modelado y validacion

Se compararon tres modelos: regresion lineal (referencia), HistGradientBoosting y Random Forest con busqueda de hiperparametros. La separacion entre entrenamiento y validacion es por motor (GroupShuffleSplit con 20% de motores y semilla fija 42), y la validacion cruzada usa GroupKFold por unit_number, de modo que ninguna fila de un motor de validacion aparece en el entrenamiento. Dividir al azar por filas habria filtrado informacion del mismo motor y dado metricas optimistas.

El mejor modelo fue **RandomForest_tuned** con los hiperparametros: max_depth = 12, min_samples_leaf = 3 y n_estimators = 300. Se entreno con 100 motores y se valido con 20.

Modelo	MAE	RMSE	R2	NASA score
RandomForest_tuned	12.29	16.97	0.834	32411
LinearRegression	15.12	19.88	0.773	27634
HistGradientBoosting	12.32	16.99	0.834	32269
Test oficial (RF)	12.85	17.93	0.814	905

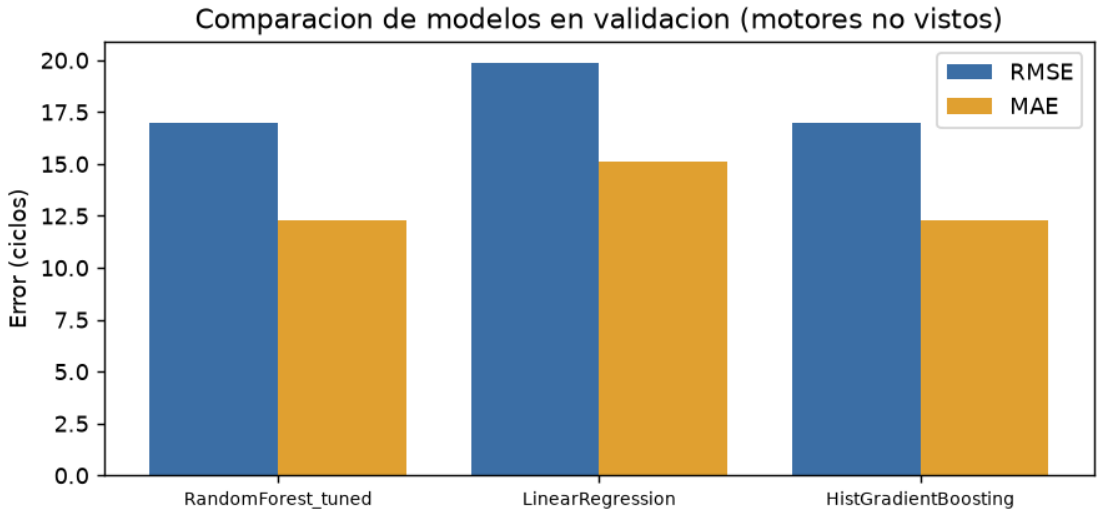


Figura 4. Error de cada modelo en validacion.

6. Resultados

En la evaluacion oficial, que usa la ultima observacion de cada uno de los 100 motores de prueba contra la RUL real de NASA, el modelo obtuvo MAE = 12.85 ciclos, RMSE = 17.93 ciclos y R2 = 0.814. En validacion el RMSE fue 16.97 y el R2 0.834. Que ambos conjuntos den metricas similares indica que no hay sobreajuste evidente.

El Random Forest mejora a la regresion lineal (RMSE 19.88) y es practicamente equivalente a HistGradientBoosting; esta ultima diferencia es demasiado pequena para declarar un ganador claro.

Sobre el NASA score. Esta metrica penaliza mas la sobreestimacion de la vida (predecir que queda mas vida de la real) que la subestimacion. No es comparable entre validacion y test, porque la validacion suma el error sobre todas las filas de 20 motores y el test solo sobre la ultima observacion de 100 motores. Ademas, la regresion lineal obtiene un NASA score menor que el Random Forest pese a su peor RMSE, lo que sugiere un sesgo distinto en el signo del error; esto debe verificarse con los residuos.

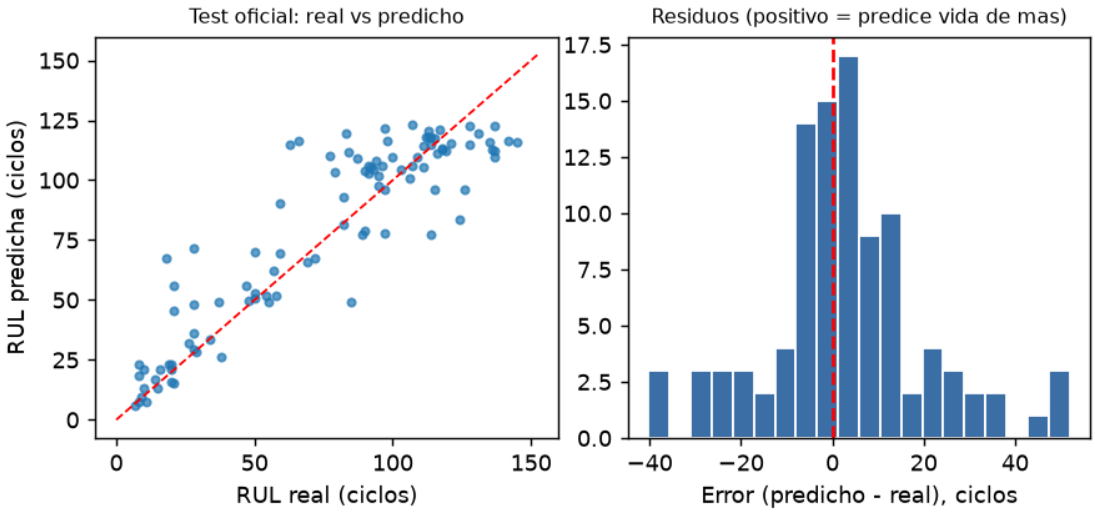


Figura 5. Evaluacion oficial: RUL real frente a predicha y distribucion de residuos.

7. Recomendaciones operativas de ingenieria

Las recomendaciones siguientes se derivan de los errores medidos. Son propuestas a validar con el area de mantenimiento, no procedimientos aprobados.

1. Alerta temprana con margen de error. Como el error tipico (RMSE) es de 18 ciclos, conviene emitir la alerta cuando la RUL predicha caiga por debajo de la RUL critica deseada mas un margen de aproximadamente dos RMSE (cerca de 36 ciclos). Asi se reduce el riesgo de actuar tarde por una sobreestimacion del modelo.

2. Inspeccion segun condicion. Programar una inspeccion boroscopica de las secciones calientes cuando la RUL predicha entre en la zona de alerta, en lugar de hacerlo a intervalo fijo, y usar el hallazgo para confirmar o descartar la prediccion.

3. Priorizar la flota por RUL. Ordenar los motores por RUL predicha permite concentrar inspecciones y repuestos en los que estan mas cerca de la falla. El tablero entregado permite explorar esa informacion por motor.

4. Vigilar la tendencia, no el dato aislado. Un valor puntual de sensor puede ser ruido. Los sensores con mayor asociacion (sensor_11, sensor_4, sensor_12, sensor_7) deben seguirse como tendencia a lo largo de los ciclos.

5. Tratar con cautela las predicciones lejanas. Al limitar la RUL a 125 ciclos, el modelo no distingue entre motores con mucha vida restante. Debe usarse para decidir cuando un motor se acerca a la falla, no para estimar su vida total.

8. Limitaciones

C-MAPSS es un conjunto sintetico con una sola condicion operativa y un solo modo de falla; el desempeno en motores reales, con varias condiciones y fallas, puede ser distinto. Los sensores se identifican por numero y no se asignan a un componente fisico concreto, porque esa asociacion debe tratarse como hipotesis de ingenieria y no como hecho confirmado. No se aplicaron conversiones de unidades por falta de unidades verificadas. Finalmente, se evaluo un solo conjunto de prueba, por lo que las metricas tienen incertidumbre que no se cuantifico.

9. Reproducibilidad

El repositorio contiene los datos originales (data/raw), el pipeline (src/pipeline.py), el notebook (notebooks/proyecto_corregido.ipynb), el modelo serializado (models), las metricas (reports/metrics.json), el tablero (dashboard.py) y el archivo requirements.txt. Se ejecuta con: python src/pipeline.py, streamlit run dashboard.py y python src/reporte.py. Las semillas aleatorias estan fijadas (random_state = 42).